

Habitat-GS: A High-Fidelity Navigation Simulator with Dynamic Gaussian Splatting

Ziyuan Xia¹, Jingyi Xu¹, Chong Cui¹, Yuanhong Yu¹, Jiazhao Zhang²,
Qingsong Yan³, Tao Ni⁴, Junbo Chen⁴, Xiaowei Zhou¹, Hujun Bao¹,
Ruizhen Hu⁵, and Sida Peng¹

¹ Zhejiang University, Hangzhou, China

² Peking University, Beijing, China

³ XGRIDS, Shenzhen, China

⁴ UDeer AI, Hangzhou, China

⁵ Shenzhen University, Shenzhen, China

Abstract. Training embodied AI agents depends critically on the visual fidelity of simulation environments and the ability to model dynamic humans. Current simulators predominantly rely on mesh-based rasterization, for which photorealistic assets are costly to author at scale, and their support for dynamic human avatars is largely constrained to mesh representations, hindering agent generalization to human-populated real-world scenarios. We present Habitat-GS, a navigation-centric embodied AI simulator extended from Habitat-Sim that integrates 3D Gaussian Splatting scene rendering and drivable gaussian avatars while maintaining full compatibility with the Habitat ecosystem. Our system implements a 3DGS renderer for real-time photorealistic rendering and supports scalable 3DGS asset import from diverse sources. For dynamic human modeling, we introduce a gaussian avatar module that enables each avatar to simultaneously serve as a photorealistic visual entity and an effective navigation obstacle, allowing agents to learn human-aware behaviors in realistic settings. Experiments on point-goal navigation demonstrate that agents trained on 3DGS scenes achieve stronger cross-domain generalization. Evaluations on avatar-aware navigation further confirm that gaussian avatars enable effective human-aware navigation, while performance benchmarks validate the system’s scalability. Code is available at <https://github.com/zju3dv/habitat-gs>.

Keywords: Embodied AI Simulation · 3D Gaussian Splatting · Dynamic Gaussian Avatar

1 Introduction

Consider a household service robot tasked with navigating to a target location in a living room where people walk, sit, and move about freely. To navigate safely and effectively, the robot must perceive its surroundings with sufficient visual

 Corresponding author. Email: pengsida@zju.edu.cn

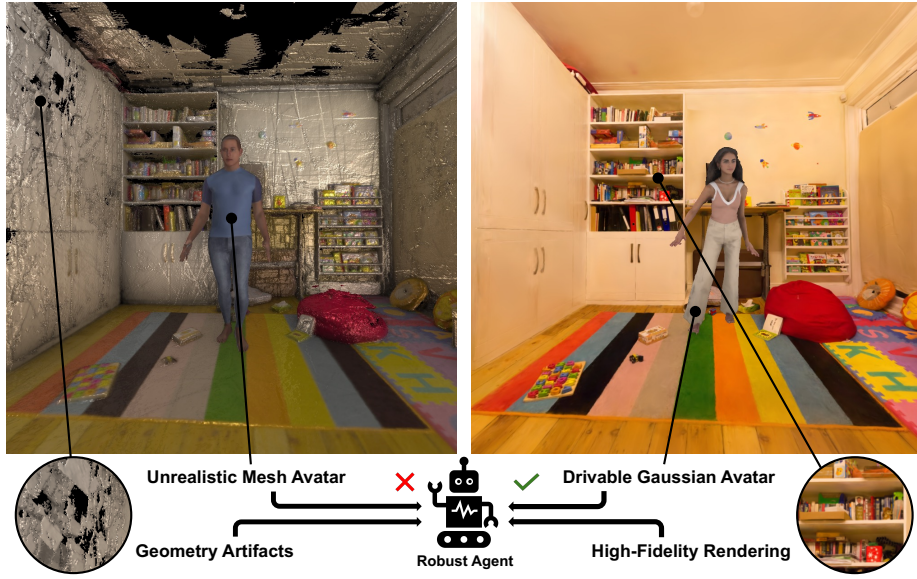


Fig. 1: Habitat-GS is a navigation-centric embodied simulation platform with 3DGS and dynamic gaussian avatars. Compared to traditional mesh-based simulators (left), our 3DGS-based simulator (right) preserves high-frequency visual details and view-dependent effects, while gaussian avatars provide realistic and dynamic human presence for human-aware navigation scenarios.

fidelity to recognize scenes and objects, and must detect and respond to dynamic human occupants for socially compliant navigation. The prevailing paradigm in Embodied AI is to train agents at scale in simulation and transfer the learned policies to reality [1, 4, 36]. This paradigm places two critical demands on the simulator: the visual realism of rendered sensor observations directly governs the effectiveness of Sim-to-Real policy transfer, and the ability to populate scenes with realistic dynamic humans determines whether agents can learn to navigate safely among people.

The leading open-source Embodied AI simulators, including Habitat-Sim [27, 30, 34], iGibson [31], AI2-THOR [13], ThreeDWorld [7], and SAPIEN [38], rely on mesh-based rasterization for visual rendering. While computationally efficient and suited for basic geometric reasoning, the mesh assets used in these simulators are rarely photorealistic, as authoring high-fidelity textured meshes is labor-intensive. The resulting visual domain gap between simulation and reality, commonly termed the Sim-to-Real gap, is an important factor impacting how well learned navigation policies transfer to physical platforms [11]. Beyond rendering quality, native support for dynamic high-fidelity human avatars remains limited across these platforms. Habitat 3.0 [27] introduced mesh-based humanoid avatars for studying social human-robot tasks, yet the visual quality of mesh avatars remains constrained. Most other simulators provide little or no

avatar support, precluding research on navigation in human-populated environments. Finally, constructing high-quality textured mesh assets from real-world data entails labor-intensive 3D scanning, artist cleanup, and semantic annotation [33], resulting in a scalability bottleneck for expanding the diversity of training environments.

Industrial-grade platforms such as NVIDIA Isaac Sim [22], built on the Omniverse rendering stack, have recently incorporated 3D Gaussian Splatting (3DGS) [12] for enhanced visual fidelity. However, their closed-source rendering backends impede the deep customization often required for research purposes. Moreover, these platforms depend on RT Core hardware available only in RTX-series GPUs, limiting their utility on datacenter-class accelerators such as A100 and H100.

We present **Habitat-GS**, a navigation-centric embodied AI simulator upgraded from Habitat-Sim that integrates 3DGS scene rendering and drivable gaussian avatars while maintaining full compatibility with the Habitat ecosystem, including Habitat-Lab tasks, training, and evaluation APIs. Habitat-GS addresses the limitations described above along three complementary axes: a 3DGS renderer for real-time photorealistic sensing within the Habitat pipeline, a drivable gaussian avatar module that serves simultaneously as a photorealistic human and an effective navigation obstacle, and scalable import of 3DGS assets from self-reconstructions, public 3DGS datasets [24, 32], and generative pipelines such as Marble [37], substantially lowering the barrier to obtaining high-quality scene assets. We validate Habitat-GS through realism studies, performance benchmarks and multiple navigation paradigms.

In summary, our contributions are as follows:

- A **high-fidelity 3DGS-based embodied AI simulation platform** extended from Habitat-Sim for navigation tasks, enabling real-time photorealistic rendering and supporting scalable import from diverse 3DGS asset sources. The platform is fully open-source and maintains complete compatibility with the Habitat ecosystem.
- A **real-time drivable gaussian avatar module** that integrates photorealistic gaussian avatar rendering with collision properties, supporting natural motion and navigation-level obstacle avoidance for human-aware navigation tasks.
- **Comprehensive experimental validation** demonstrates, via VLM-based assessment and PointNav experiments, that 3DGS rendering improves agent cross-domain generalization through a mixed-domain training strategy, and that gaussian avatars enhance agent capability in human-populated scenarios, accompanied by performance benchmarks confirming system scalability.

2 Related Work

2.1 Embodied AI Simulators

Embodied AI research relies on simulation environments for large-scale parallel training and subsequent Sim-to-Real policy transfer [30, 36]. We compare existing

Table 1: Comparison of Embodied AI simulation platforms. “GPU Req.” indicates hardware requirements beyond standard CUDA-capable GPUs. “No Special Req.” means standard GPUs are sufficient. Habitat-GS is the only platform combining 3DGS rendering with drivable gaussian avatars while remaining fully open-source and deployable on standard datacenter accelerators.

Platform	Render Asset	Humanoid Avatar	Open-Source	GPU Req.
Habitat [27, 30, 34]	Mesh	Mesh (SMPL-X)	✓	No Special Req.
iGibson [15, 31]	Mesh	Mesh (Rigid-body)	✓	No Special Req.
AI2-THOR [13]	Mesh	Mesh (URDF)	✓	No Special Req.
ThreeDWorld [7]	Mesh	Mesh (Replicants)	✓	No Special Req.
SAPIEN [38]	Mesh	Mesh (URDF)	✓	No Special Req.
Isaac Sim [22]	Mesh+3DGS	Mesh (Sim-Ready)	Partial	Need RT Cores
Habitat-GS (Ours)	Mesh+3DGS	Mesh+GS (SMPL-X)	✓	No Special Req.

platforms along four dimensions, namely *render asset type*, *humanoid avatar support*, *platform openness*, and *hardware requirements*, in Tab. 1.

The predominant embodied AI simulators, including Habitat-Sim [27, 30, 34], iGibson [15, 31], AI2-THOR [13], ThreeDWorld [7], and SAPIEN [38], all employ mesh-based rasterization for scene rendering. Driving simulators such as CARLA [6] confront the same rendering-fidelity challenge, and are increasingly augmented with neural rendering. Regarding humanoid avatars, while some platforms now provide some form of mesh-based representation, such as URDF-driven articulated bodies [13, 38], rigid-body tracks [15], and Unity Replicants [7], these uniformly lack the visual fidelity needed for training agents to perceive fine-grained human appearance and motion cues. Habitat 3.0 [27] offers the most capable open-source avatar system with deformable SMPL-X mesh avatars for human-robot social tasks, yet mesh rendering inherently constrains visual quality.

Habitat-GS inherits the Habitat ecosystem’s high-performance infrastructure and training APIs while upgrading the rendering pipeline from mesh to 3DGS and natively integrating drivable gaussian avatars, uniquely combining photo-realistic rendering, dynamic high-fidelity humanoids, and a mature open-source research ecosystem.

2.2 Neural Rendering

Neural Radiance Fields (NeRF) [20] demonstrated that implicit neural scene representations can synthesize photorealistic novel views from multi-view images. Subsequent works significantly improved training speed and rendering quality [3, 21]. However, NeRF’s volume rendering paradigm requires per-pixel ray marching, yielding frame rates far below the real-time requirements of embodied AI sensor pipelines. Moreover, its implicit representation is difficult to integrate with the explicit rendering backends employed by existing simulators.

3D Gaussian Splatting (3DGS) [12] represents scenes as collections of explicit anisotropic 3D Gaussians and renders them via differentiable tile-based

rasterization, achieving real-time frame rates while maintaining rendering quality comparable to NeRF. The explicit nature of 3DGS lends itself naturally to integration with traditional graphics pipelines, for example via CUDA-OpenGL interop, and facilitates spatial editing, asset composition, and dynamic deformation. Follow-up works have extended 3DGS along multiple directions, including anti-aliasing [39] and dynamic scene modeling [19], further broadening its applicability.

2.3 Gaussian Avatars

Parametric body models such as SMPL [18] and SMPL-X [25] provide differentiable mappings from pose parameters to human body meshes, serving as the standard infrastructure for avatar animation. Building on this foundation, a growing number of work combines 3DGS with parametric body models to create high-fidelity, drivable human avatars. AnimatableGaussians [16] maps 2D Gaussians into UV space for pose-driven rendering. GaussianAvatar [9] deforms canonical-space gaussians to posed configurations via Linear Blend Skinning (LBS). GART [14] introduces gaussian articulated templates with differentiable deformation. HumanGaussian [17] leverages Score Distillation Sampling [26] for text-driven gaussian avatar generation. The shared paradigm across these methods defines gaussian attributes in a canonical space and transforms them to target poses via skinning. This approach achieves a favorable balance between visual quality and rendering speed that surpasses mesh-based avatars with limited geometric fidelity and NeRF-based avatars with insufficient rendering speed.

3 Habitat-GS Simulation Environment

Habitat-GS extends Habitat-Sim with two key upgrades: extending mesh-based scene rendering with 3D Gaussian Splatting (Sec. 3.2), and implementing a gaussian avatar module to drive and render avatars (Sec. 3.3). A central design principle underlying both modules is *visual-navigation decoupling*: 3DGS handles all visual rendering and traditional NavMeshes continue to govern navigation, which sidesteps the lack of explicit surface geometry inherent in the 3DGS representation. Below we discuss the overall system architecture and the detailed design of each module.

3.1 System Overview

Fig. 2 illustrates the end-to-end data flow in three stages:

Asset Preparation. The scene pipeline prepares 3DGS assets in standard PLY format, together with the conventional NavMesh required by Habitat-Sim’s navigation logic. The avatar pipeline produces three artifacts per identity: *first*, canonical gaussian attributes, specifically positions, SH coefficients, opacities, scales, rotations, and LBS weights, exported from a trained gaussian avatar

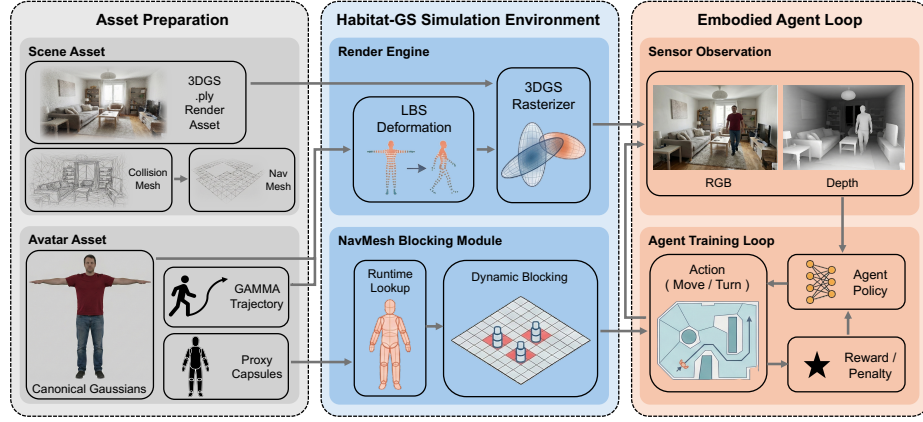


Fig. 2: System overview of Habitat-GS. From left to right: *Asset Preparation*, where 3DGS scene assets and gaussian avatar assets are prepared respectively. *Habitat-GS Simulation Environment*, where the render engine performs 3DGS rasterization for scene gaussians and LBS deformation followed by rasterization for avatar gaussians, producing RGB-D observations. The NavMesh blocking module retrieves pre-computed proxy capsules at runtime and injects them into the NavMesh for obstacle blocking. *Embodied Agent Loop*, where the agent policy consumes sensor observations, executes actions, and receives rewards shaped by both navigation success and avatar collision.

model. *Second*, GAMMA-generated [40] motion trajectories converted to per-frame SMPL-X joint transformation matrices. *Third*, proxy capsules that approximate the avatar’s per-frame collision geometry.

Habitat-GS Simulation Environment. At each frame, the system operates two parallel modules. The *render engine* dispatches scene gaussians to the CUDA rasterizer and simultaneously deforms avatar gaussians to the current pose via CUDA Linear Blend Skinning before rasterizing them with an independent renderer instance. Both color and depth outputs are transferred to OpenGL textures through CUDA–OpenGL interop. The *navigation module* retrieves the pre-computed proxy capsules for current frame and injects them into the NavMesh as dynamic obstacles, enabling the path planner to produce collision-aware steps around moving avatars.

Embodied Agent Loop. The composited RGB-D observations serve as sensor inputs to the agent policy, which outputs discrete actions including move forward, turn left/right, and stop. The augmented NavMesh ensures that actions violating avatar collision boundaries are clipped, while two avatar state query APIs expose clearance distances and collision flags for reward shaping and metric computation in downstream tasks (Sec. 3.4).

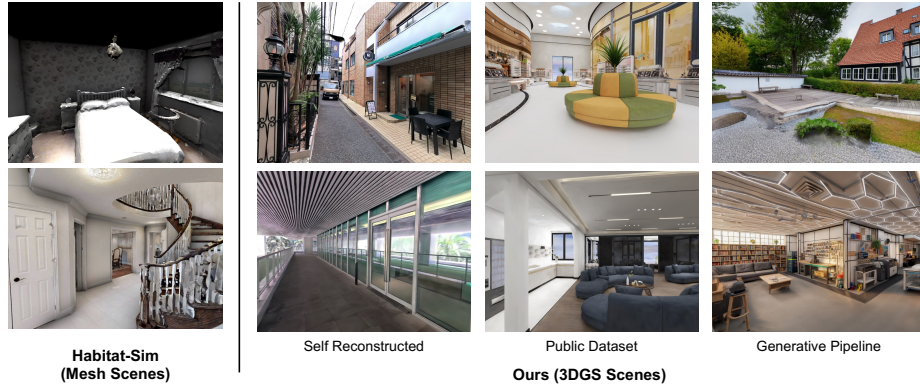


Fig. 3: Visual comparison of scene rendering. Mesh-based rendering (left) vs. our 3DGS rendering (right). Our simulator is based on 3DGS, which preserves high-frequency details and supports diverse sources of rendering assets.

3.2 3DGS Scene Rendering Integration

The core technical challenge in integrating 3DGS into Habitat-Sim lies in bridging two heterogeneous rendering backends: the Habitat sensor pipeline is built on OpenGL, whereas high-performance 3DGS rasterization relies on CUDA tile-based rendering [12]. We address this with a zero-copy CUDA–OpenGL interoperability mechanism that keeps all rendering data on the GPU. The CUDA rasterizer writes color and depth directly into pre-registered OpenGL textures, eliminating CPU-side transfer and enabling real-time photorealistic rendering with minimal overhead. As shown in Fig. 3, this yields markedly higher visual fidelity than mesh-based rendering. We detail this *Map–Render–Unmap* mechanism in the supplementary material.

A Habitat-GS scene may mix 3DGS assets, traditional meshes, and multiple gaussian avatars. To resolve occlusion among them, a full-screen pass composites the CUDA-produced 3DGS depth with the OpenGL depth buffer. For each pixel the GS pass yields (C_g, D_g) and the mesh pass (C_m, D_m) , and the shader keeps the nearer fragment, $(C, D) = (D_g < D_m) ? (C_g, D_g) : (C_m, D_m)$. All active avatars’ deformed gaussians are rasterized in a single CUDA pass and composited likewise, giving correct depth ordering among scenes, meshes, and an arbitrary number of avatars within one frame.

3.3 Dynamic Gaussian Avatar Module

Existing gaussian avatar methods [9, 16] focus exclusively on standalone reconstruction and rendering, without coupling to simulator sensor pipelines or navigation mesh systems. To address this limitation, Habitat-GS incorporates an avatar module that supports the rendering, driving, and collision detection of gaussian avatars, which render with markedly higher visual fidelity than mesh

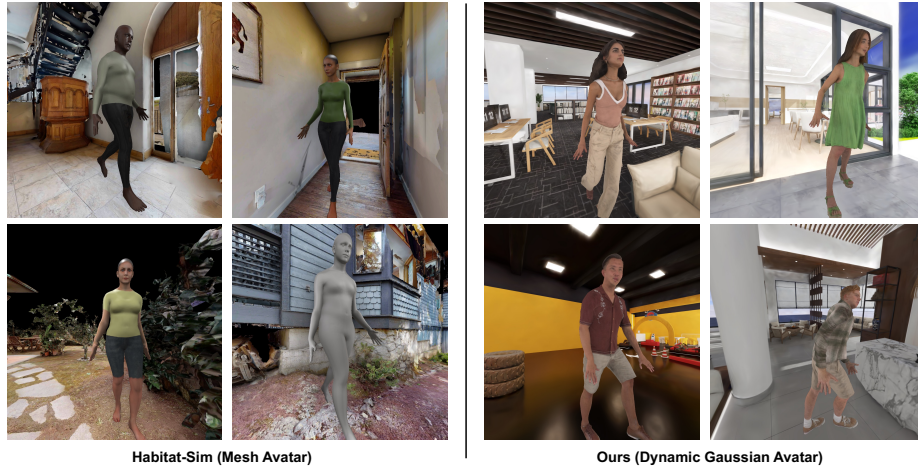


Fig. 4: Qualitative comparison of mesh avatars and gaussian avatars. Gaussian avatars exhibit significantly higher visual fidelity, preserving fine-grained details such as clothing wrinkles and hair texture.

avatars (Fig. 4). We pre-bake canonical gaussian attributes offline and employ a lightweight CUDA LBS kernel to deform them to arbitrary SMPL-X [25] poses, thereby avoiding costly neural network inference at runtime. Below we detail how this rendering capability is combined with driving trajectory and NavMesh blocking mechanism.

GAMMA Trajectory. To drive avatars along scene-aware, physically plausible walking paths, we run the GAMMA motion generation model [40] offline on NavMesh shortest paths through sampled waypoints, and convert the resulting poses into per-frame SMPL-X joint transformation matrices via forward kinematics. At runtime, the avatar module interpolates these matrices at the current simulation time and passes them to the CUDA LBS kernel for deformation.

Dynamic NavMesh Blocking. Since the 3DGS representation lacks a well-defined geometric surface, it is unsuitable for direct collision detection. We therefore decouple appearance from collision through a *proxy capsule* mechanism. During the offline stage, we derive capsule primitives from the SMPL-X skeletal bones and pre-compute their per-frame world-space positions. At runtime, these are retrieved by temporal interpolation at negligible cost. On the NavMesh side, to capture the time-varying occupancy of moving avatars, we extend the Habitat-Sim path planner with *dynamic capsule obstacles*. At each step the active avatars’ capsules are injected into the planner, which clips any agent motion that would intersect a capsule and thereby guarantees the agent never passes through an avatar’s body. The full avatar kinematics, CUDA LBS, and capsule pre-computation/injection pipelines are detailed in the supplementary material.

3.4 APIs for Habitat-Lab

Habitat-GS is designed for seamless integration with Habitat-Lab. During scene initialization, the simulator automatically loads gaussian avatar configurations from the scene description file and instantiates all avatars. At each simulation step, avatar pose updates are triggered transparently, synchronizing both visual and navigation pipelines. The sensor outputs, both RGB and depth, produced by 3DGS rendering are format-identical to those of the mesh renderer, allowing existing Habitat-Lab tasks including PointNav [1] to run on 3DGS scenes without modification.

For avatar-aware navigation tasks such as avatar-aware point-goal navigation, two additional query APIs are provided. One returns the distance to the nearest avatar capsule, and the other indicates whether a candidate step is blocked. Together, these provide the information needed for computing rewards and metrics. Researchers can leverage these APIs to design proximity-based rewards, collision penalties, and tracking metrics within the Habitat-Lab framework.

4 Experiments

We evaluate Habitat-GS along three complementary axes: *First*, we demonstrate that 3DGS scenes are more photorealistic than mesh scenes through VLM, human, and feature-space assessments (Sec. 4.2), and that training on these higher-quality scenes improves agent cross-domain generalization and Sim-to-Real transfer on PointNav (Sec. 4.3). *Second*, we examine whether training with dynamic gaussian avatars equips agents with human-aware navigation capabilities on dynamic point-goal navigation (Sec. 4.4). *Third*, we examine whether the system remains efficient under varying scene complexity and avatar counts (Sec. 4.5). We first describe the shared experimental setup.

4.1 Experimental Setup

Datasets. For *3DGS scenes*, we combine the InteriorGS dataset [32] with additional real-world reconstructed GS scenes in a 4:1 ratio, yielding 120 scenes split into 100 for training and 20 for testing. For *mesh scenes*, we use the Habitat-Matterport 3D (HM3D) dataset [29], similarly split into 100 training and 20 test scenes. Importantly, the GS and mesh test sets are drawn from disjoint scene collections rather than two representations of the same physical spaces, so that evaluation measures cross-domain generalization.

For *gaussian avatars*, we export canonical gaussians from six trained AnimateGaussians [16] identities, with three for training and three held out for testing. Each avatar is driven by GAMMA-generated [40] motion trajectories with pre-computed joint matrices and proxy capsules.

Evaluation Metrics. We adopt standard embodied navigation metrics. **Success Rate (SR)** measures the fraction of episodes in which the agent reaches

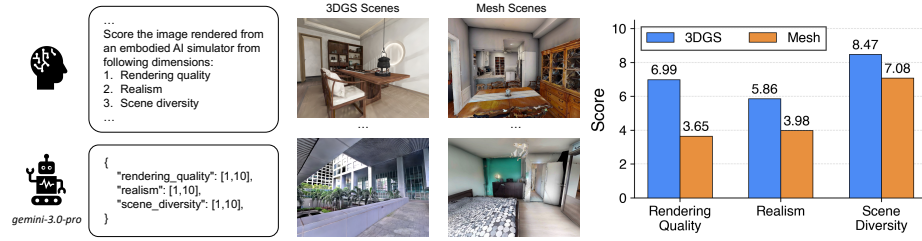


Fig. 5: VLM scene quality assessment. Gemini 3.0 Pro evaluates 240 rendered screenshots from each domain on three perceptual dimensions. GS scenes consistently outperform mesh scenes, confirming their superior visual fidelity and diversity.

the goal within a distance threshold. **Success weighted by Path Length (SPL)** [1] jointly captures success and path efficiency: $SPL = SR \times (\ell^*/\ell)$, where ℓ^* is the geodesic shortest-path length and ℓ the agent’s actual path length. **Distance to Goal (DTG)** records the Euclidean distance between the agent and the goal at episode termination. Lower values indicate the agent consistently navigates closer to the target even in unsuccessful episodes.

For avatar-aware tasks, we additionally report: **Collision Rate (CR)**, the fraction of collision steps. **Personal Space Intrusion (PSI)**, which quantifies the average degree to which the agent enters the 1.0m personal-space radius around each avatar. Higher values indicate more frequent and severe intrusions.

4.2 3DGS Scenes Deliver Higher Quality Rendering

We assess the rendering quality of 3DGS scenes and mesh scenes from three complementary angles: an automated VLM judge, a study with human raters, and a quantitative comparison to real photographs in feature space.

VLM Scene Quality Assessment. To objectively quantify the quality gap between 3DGS and mesh scene rendering, we employ Gemini 3.0 Pro [35] as an automated evaluator. We render 240 screenshots evenly from each renderer, divided into 48 evaluation batches, each containing 5 GS and 5 mesh images with randomized indices to blind the model of rendering source. The VLM scores each image on a 10-point scale along three dimensions: *rendering quality*, *realism*, and *scene diversity*. As shown in Fig. 5, GS scenes score higher than mesh scenes across all three dimensions in this automated assessment.

Human Perceptual Study. We next corroborate the evaluation with a study using human raters. Each batch presents 3 GS and 3 mesh renderings in randomized order, and raters are asked to rank them by perceived realism. Across 88 participants and 825 valid batches with 7425 GS-vs-mesh pairwise comparisons, GS renderings attain a markedly better mean rank and are preferred in 61.9% of pairwise comparisons, which strongly supports the realism advantage of 3DGS rendering.

Table 2: Realism evaluation of 3DGS vs. mesh rendering. (a) Human ranking study. **(b)** Distance of GS / mesh renderings to a pool of real photographs in CLIP and DINOv2 feature spaces. **Best** (more realistic / closer to real) is highlighted.

(a) Human perceptual ranking			(b) Feature-space distance to real photos ($\downarrow$)				
	GS	Mesh		CLIP		DINOv2	
			Real reference & metric	GS	Mesh	GS	Mesh
Mean rank ($\downarrow$)	3.14	3.86	InCrowd-VI [2] / cos	0.252	0.343	0.583	0.729
Pairwise wins ($\uparrow$)	61.9%	38.1%	InCrowd-VI [2] / CMMD	0.131	0.186	0.108	0.148
			Self-captured / cos	0.155	0.256	0.470	0.772
			Self-captured / CMMD	0.068	0.123	0.052	0.104

Feature-Space Realism Evaluation. As an objective complement, we measure how close each renderer is to *real* photographs in learned feature spaces. Using 240 GS and 240 mesh renderings and a pool of 480 real photos (240 InCrowd-VI [2] and 240 self-captured), we compute centroid cosine distance and CMMD [10] in CLIP [28] and DINOv2 [23] feature spaces, where a smaller distance means closer to real. As shown in Tab. 2(b), GS renderings are closer to real photographs than mesh renderings on every backbone, metric, and reference pool. Taken together, the VLM, human and feature-space assessments all give consistent evidence that 3DGS scenes deliver more realistic and higher quality rendering than mesh scenes under comparable asset-acquisition effort.

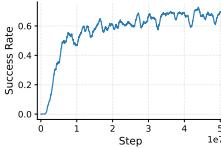
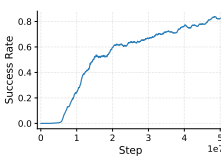
4.3 3DGS Scenes Train More Generalizable Agents

We then test whether the rendering-quality advantage yields navigation agents that generalize better, at two levels: across simulation domains (Sim-to-Sim) and from simulation to real-world video (Sim-to-Real).

Sim-to-Sim PointNav Evaluation. We train five agent groups under different scene-domain mixtures on the **PointNav** [1] task, with training budget fixed at 5×10^7 steps: **A**: 100 mesh scenes, **B**: 100 GS scenes, **C**: 80 M + 20 G, **D**: 50 M + 50 G, and **E**: 20 M + 80 G. All agents share a unified DD-PPO [36] architecture with a ResNet [8] visual encoder and a GRU [5] policy head, receiving 256×256 RGB and depth observations, with only training scene composition varying. Each agent is evaluated on both the 20-scene mesh test set and the 20-scene GS test set, so that performance on the unseen domain measures cross-domain generalization.

Sim-to-Real Validation on Real-World Videos. To further evaluate the robustness of agents trained on different domains, we run an open-loop PointNav evaluation on real videos: 5 self-recorded scenes (Record3D / iPhone, 38 episodes) and 5 InCrowd-VI [2] scenes (Aria glasses, 25 episodes). At each frame

Table 3: PointNav results and training curves. (a) Results across different training domain mixtures. Agents are trained on varying ratios of mesh (M) and 3DGS (G) scenes and evaluated on disjoint mesh and GS test sets. SR and SPL are in %; DTG is in meters ($\downarrow$). **Best** and **Second best** results per column are highlighted. (b) Training success rate curve for 100 Mesh (Config A). (c) Training success rate curve for 100 GS (Config B).

(a) Quantitative Results							(b) Mesh Training
Training Config	Mesh Test			GS Test			
	SR $\uparrow$	SPL $\uparrow$	DTG $\downarrow$	SR $\uparrow$	SPL $\uparrow$	DTG $\downarrow$	
A: 100 Mesh	59.00	51.23	5.537	61.30	52.09	4.982	
B: 100 GS	53.00	43.13	6.439	70.70	58.49	3.550	
C: 80 M + 20 G	60.20	52.06	6.004	73.40	64.32	3.757	
D: 50 M + 50 G	61.80	51.34	5.938	78.10	67.42	3.008	
E: 20 M + 80 G	59.60	51.01	5.901	79.60	68.38	2.698	

we compare the trained agent’s discrete action to the human’s actual next motion and report action-prediction agreement with a ± 1 -step tolerance (APA ± 1). This open-loop protocol probes whether the agent’s predicted actions align with real human trajectories, testing the Sim-to-Real transfer capability.

Analysis. Tab. 3 and Tab. 4 reveal a clear picture of three training strategies:

Mesh-only training converges quickly but generalizes poorly. As seen in Tab. 3(b), training on 100 mesh scenes converges rapidly, yet the resulting agent plateaus at a modest success rate around 0.6 and generalizes poorly to the GS test set. The same ranking holds under Sim-to-Real in Tab. 4. The mesh-only agent attains the lowest real-video APA ± 1 on both sources, confirming that it transfers least well to reality. Mesh-only training therefore generalizes least effectively to realistic visual domains within this budget.

GS-only training builds stronger capability but converges too slowly. Tab. 4 shows that training on 3DGS scenes can substantially improve performance on Sim-to-Real evaluation compared with mesh-only training, proving that the added visual realism aids real-world transfer. However, because GS scenes are more realistic and diverse, the agent requires considerably more steps to converge. Within the fixed budget of 5×10^7 steps, the GS-only agent has not fully converged according to Tab. 3(c), and its mesh test performance remains below that of Config A. Pure GS training is therefore not cost-effective under practical compute constraints.

Mixed-domain training is the most effective strategy. Configurations D (50 M + 50 G) and E (20 M + 80 G) demonstrate the most effective approach: a small

Table 4: Real-video open-loop PointNav Evaluation. At each frame we compare the trained agent’s discrete action to the human’s actual next motion and report $\text{APA} \pm 1$ (%). **N** is the number of episodes per scene. **Best**, **2nd**, and **3rd** in each scene column are highlighted. The *mixed* > *GS* > *mesh* ordering on real video mirrors the cross-domain trend in Tab. 3.

Scene N	Self-recorded (Record3D, iPhone)						InCrowd-VI (Aria glasses)					
	yard	lab.	gard.	off.	park.	Avg	Zur.	BIN	UZH	TS	AND	Avg
	9	9	2	9	9	38	3	4	8	4	6	25
A: 100 Mesh	66.2	28.6	78.5	61.0	61.4	55.6	33.8	11.5	8.4	47.9	17.1	20.4
B: 100 GS	78.1	25.2	78.5	62.6	61.8	58.1	44.4	18.8	16.9	41.1	24.1	26.1
C: 80 M + 20 G	76.1	24.5	74.0	62.1	69.2	58.8	70.3	13.3	9.7	41.4	17.2	24.4
D: 50 M + 50 G	79.3	29.2	77.0	65.7	61.7	59.9	71.5	12.5	15.3	60.3	18.8	29.6
E: 20 M + 80 G	81.6	23.7	75.0	64.3	72.0	61.2	73.5	17.5	12.4	45.6	22.4	28.3

number of mesh scenes first establish foundational navigation competence, after which training on more realistic GS scenes further strengthens the agent’s ability to handle diverse and realistic environments. Config **E** achieves the best GS test performance while maintaining mesh test performance on par with the mesh-only baseline. The same advantage carries over to Sim-to-Real in Tab. 4: the mixed-domain agents attain the best real-video $\text{APA} \pm 1$ overall, with Config **E** reaching 61.2 on the self-recorded set and Config **D** reaching 29.6 on InCrowd-VI. This offers convincing evidence that 3DGS training helps narrow the visual Sim-to-Real gap. These results together indicate that the two data modalities are complementary: mesh scenes provide efficient geometric learning, while GS scenes contribute the visual realism that transfers to more realistic deployment environments.

4.4 Gaussian Avatars Equip Agents with Human-Aware Navigation

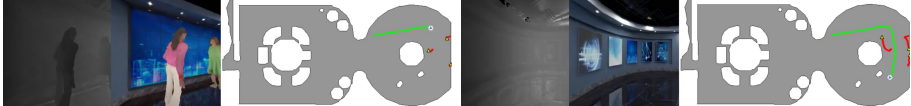
We next evaluate whether gaussian avatars enable agents to navigate safely among dynamic humans. We extend the standard PointNav task by introducing three gaussian avatars walking around in scenes, and agents are supposed to reach the goal point without colliding with moving avatars.

The agents are first trained on static PointNav for 5×10^7 steps to establish basic navigation competence, and then fine-tuned for an additional 5×10^6 steps on a subset of 20 GS scenes under two configurations: **Baseline**: GS scenes, no avatars; and **GS Scene + GS Avatar**: GS scenes with gaussian avatars. Both groups are evaluated on two dynamic test sets to assess cross-domain capability: 20 mesh scenes with mesh avatars, and 20 GS scenes with gaussian avatars. We prioritize **CR** and **PSI** in avatar-aware navigation evaluation, for collision is unacceptable in real-world human-aware navigation.

Analysis. Tab. 5 demonstrates the clear benefit of training with gaussian avatars for human-aware navigation. *First*, compared to the avatar-free base-

Table 5: Avatar-aware PointNav results. Top: Two training configurations are evaluated: a GS scene baseline without avatars, and a GS scene populated with GS avatars. Agents are evaluated on both mesh and GS dynamic test sets to assess in-domain performance and cross-domain generalization. SR, SPL and CR are in %. CR is the fraction of collision steps ($\downarrow$). PSI is the average personal-space intrusion ($\downarrow$). **Bottom:** Visual demonstration of our avatar-aware navigation policy. Agent successfully navigates through gaussian avatars without collision or personal space intrusion.

Training Config	Mesh Scene+Avatar Test				GS Scene+Avatar Test			
	SR $\uparrow$	SPL $\uparrow$	CR $\downarrow$	PSI $\downarrow$	SR $\uparrow$	SPL $\uparrow$	CR $\downarrow$	PSI $\downarrow$
GS Baseline	54.80	46.98	2.521	0.075	81.80	71.35	6.713	0.092
GS Scene+GS Avatar	58.00	46.80	2.342	0.068	80.00	65.72	4.746	0.077



(a) Start of episode



(b) End of episode

line, training with gaussian avatars effectively lowers the Collision Rate (CR) and Personal Space Intrusion (PSI) on the in-domain GS test set. This human-aware navigation capability is acquired with only a small number of fine-tuning steps (5×10^6 , which is only 10% of the static pre-training budget), confirming that agents efficiently learn collision avoidance behaviors from the photorealistic humans and realistic motion simulations. *Second*, the agent fine-tuned with gaussian avatars also achieves lower CR (2.342% vs. 2.521%) and PSI (0.068 vs. 0.075) on the mesh test set. This cross-domain improvement suggests that the perceptual cues learned from gaussian avatars, such as recognizing human shape, anticipating walking direction, and estimating personal space, transfer effectively even to lower-fidelity mesh environments.

4.5 Performance Benchmarks

To confirm that Habitat-GS’s photorealistic rendering does not compromise training speed significantly, we benchmark rendering speed and GPU memory under varying scene scales and avatar counts. All measurements are conducted on Habitat-Lab’s PointNav training on a single NVIDIA RTX 4090 GPU at 256×256 resolution, matching the typical training configuration.

Analysis. Tab. 6 demonstrates two key properties of the system. *First*, under typical RL training loads, Habitat-GS sustains real-time rendering sufficient for large-scale DD-PPO training. For medium-scale scenes with 1–2 avatars, the system still maintains >50 FPS, well above the threshold for efficient parallel training. *Second*, GPU memory consumption grows approximately linearly with both scene gaussian count and avatar count, providing predictable resource

Table 6: Rendering performance and GPU memory usage. Mesh Ref. reports Habitat-Sim’s native mesh rasterization on a representative HM3D scene as a baseline. (a) Varying scene Gaussian count with no avatars. (b) Varying avatar count in a medium-scale scene with approximately 2M Gaussians. Habitat-GS maintains real-time throughput under typical RL training loads with up to 5M Gaussians and <5 avatars.

	Mesh	(a) Scene Scale (Gaussian Count)							(b) Avatar Count				
	Ref.	300 K	500 K	1 M	3 M	5 M	7 M		0	1	2	5	10
FPS↑	163.8	159.2	148.8	120.9	82.60	51.46	44.52	94.16	75.14	57.70	37.74	24.67	
Frame time (ms)↓	6.11	6.28	6.72	8.27	12.1	19.4	22.5	10.6	13.3	17.3	26.5	40.5	
Mem (GB)↓	2.930	3.299	3.373	3.567	3.723	4.091	4.425	3.917	4.197	4.457	5.513	7.497	

budgeting. The throughput reduction of 3DGS rendering, relative to mesh-based rasterization, is a favorable trade-off given the improvements in visual fidelity and agent generalization demonstrated in Sec. 4.2 and Sec. 4.3.

5 Conclusion

We present Habitat-GS, a navigation-centric embodied AI simulator that upgrades the visual backbone of Habitat-Sim from mesh rasterization to 3D Gaussian Splatting, and populates environments with photorealistic drivable gaussian avatars. Habitat-GS is fully open-sourced to provide a high-fidelity, ecosystem-compatible foundation for future embodied AI research.

Limitations. A central design choice of Habitat-GS is *visual–navigation decoupling* (Sec. 3.1): 3DGS handles all visual rendering while NavMeshes are responsible for navigation logic. This intentional decoupling sidesteps the absence of explicit surface geometry in the 3DGS representation. However, it also confines physical interaction to navigation-level obstacle avoidance rather than force- or impulse-level contact. Habitat-GS is therefore best suited for navigation tasks. Extending support to manipulate GS-represented objects would require deeper integration with the physics engine, which we identify as a promising direction for future work. In addition, avatars in Habitat-GS are currently driven along pre-synthesized trajectories and act as high-priority dynamic obstacles rather than interactive co-agents. Coupling avatars to agent behavior at runtime would require neural inference in the loop and is left to future work.

Acknowledgements

This work was partially supported by NSFC (No. U24B20154, No. 62402427), Zhejiang Provincial Natural Science Foundation of China under Grant No. LZ26-F020003, “Innovation Yongjiang 2035” Key R&D Program (No. 2025Z061) and Information Technology Center and State Key Lab of CAD&CG, Zhejiang University.

References

1. Anderson, P., Chang, A., Chaplot, D.S., Dosovitskiy, A., Gupta, S., Koltun, V., Kosecka, J., Malik, J., Mottaghi, R., Savva, M., Zamir, A.R.: On evaluation of embodied navigation agents (2018), <https://arxiv.org/abs/1807.06757>
2. Bamdad, M., Hutter, H.P., Darvishy, A.: Incrowd-vi: A realistic visual-inertial dataset for evaluating simultaneous localization and mapping in indoor pedestrian-rich spaces for human navigation. *Sensors* **24**(24), 8164 (Dec 2024). <https://doi.org/10.3390/s24248164>, <http://dx.doi.org/10.3390/s24248164>
3. Barron, J.T., Mildenhall, B., Verbin, D., Srinivasan, P.P., Hedman, P.: Mip-nerf 360: Unbounded anti-aliased neural radiance fields (2022), <https://arxiv.org/abs/2111.12077>
4. Batra, D., Gokaslan, A., Kembhavi, A., Maksymets, O., Mottaghi, R., Savva, M., Toshev, A., Wijmans, E.: Objectnav revisited: On evaluation of embodied agents navigating to objects (2020), <https://arxiv.org/abs/2006.13171>
5. Cho, K., van Merriënboer, B., Gulcehre, C., Bahdanau, D., Bougares, F., Schwenk, H., Bengio, Y.: Learning phrase representations using rnn encoder-decoder for statistical machine translation (2014), <https://arxiv.org/abs/1406.1078>
6. Dosovitskiy, A., Ros, G., Codevilla, F., Lopez, A., Koltun, V.: Carla: An open urban driving simulator (2017), <https://arxiv.org/abs/1711.03938>
7. Gan, C., Schwartz, J., Alter, S., Mrowca, D., Schrimpf, M., Traer, J., Freitas, J.D., Kubilius, J., Bhandwaldar, A., Haber, N., Sano, M., Kim, K., Wang, E., Lingelbach, M., Curtis, A., Feiglis, K., Bear, D.M., Gutfreund, D., Cox, D., Torralba, A., DiCarlo, J.J., Tenenbaum, J.B., McDermott, J.H., Yamins, D.L.K.: Threedworld: A platform for interactive multi-modal physical simulation (2021), <https://arxiv.org/abs/2007.04954>
8. He, K., Zhang, X., Ren, S., Sun, J.: Deep residual learning for image recognition (2015), <https://arxiv.org/abs/1512.03385>
9. Hu, L., Zhang, H., Zhang, Y., Zhou, B., Liu, B., Zhang, S., Nie, L.: Gaussianavatar: Towards realistic human avatar modeling from a single video via animatable 3d gaussians (2024), <https://arxiv.org/abs/2312.02134>
10. Jayasumana, S., Ramalingam, S., Veit, A., Glasner, D., Chakrabarti, A., Kumar, S.: Rethinking fid: Towards a better evaluation metric for image generation (2024), <https://arxiv.org/abs/2401.09603>
11. Kadian, A., Truong, J., Gokaslan, A., Clegg, A., Wijmans, E., Lee, S., Savva, M., Chernova, S., Batra, D.: Sim2real predictivity: Does evaluation in simulation predict real-world performance? *IEEE Robotics and Automation Letters* **5**(4), 6670–6677 (Oct 2020). <https://doi.org/10.1109/LRA.2020.3013848>, <http://dx.doi.org/10.1109/LRA.2020.3013848>
12. Kerbl, B., Kopanas, G., Leimkühler, T., Drettakis, G.: 3d gaussian splatting for real-time radiance field rendering (2023), <https://arxiv.org/abs/2308.04079>
13. Kolve, E., Mottaghi, R., Han, W., VanderBilt, E., Weihs, L., Herrasti, A., Deitke, M., Ehsani, K., Gordon, D., Zhu, Y., Kembhavi, A., Gupta, A., Farhadi, A.: Ai2-thor: An interactive 3d environment for visual ai (2022), <https://arxiv.org/abs/1712.05474>
14. Lei, J., Wang, Y., Pavlakos, G., Liu, L., Daniilidis, K.: Gart: Gaussian articulated template models (2023), <https://arxiv.org/abs/2311.16099>
15. Li, C., Xia, F., Martín-Martín, R., Lingelbach, M., Srivastava, S., Shen, B., Vainio, K., Gokmen, C., Dharan, G., Jain, T., Kurenkov, A., Liu, C.K., Gweon, H., Wu, J., Fei-Fei, L., Savarese, S.: igibson 2.0: Object-centric simulation for robot learning of everyday household tasks (2021), <https://arxiv.org/abs/2108.03272>

16. Li, Z., Zheng, Z., Wang, L., Liu, Y.: Animatable gaussians: Learning pose-dependent gaussian maps for high-fidelity human avatar modeling. In: Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR) (2024)
17. Liu, X., Zhan, X., Tang, J., Shan, Y., Zeng, G., Lin, D., Liu, X., Liu, Z.: Humangaussian: Text-driven 3d human generation with gaussian splatting (2024), <https://arxiv.org/abs/2311.17061>
18. Loper, M., Mahmood, N., Romero, J., Pons-Moll, G., Black, M.J.: SMPL: A skinned multi-person linear model. *ACM Trans. Graphics (Proc. SIGGRAPH Asia)* **34**(6), 248:1–248:16 (Oct 2015)
19. Luiten, J., Kopanas, G., Leibe, B., Ramanan, D.: Dynamic 3d gaussians: Tracking by persistent dynamic view synthesis (2023), <https://arxiv.org/abs/2308.09713>
20. Mildenhall, B., Srinivasan, P.P., Tancik, M., Barron, J.T., Ramamoorthi, R., Ng, R.: Nerf: Representing scenes as neural radiance fields for view synthesis (2020), <https://arxiv.org/abs/2003.08934>
21. Müller, T., Evans, A., Schied, C., Keller, A.: Instant neural graphics primitives with a multiresolution hash encoding. *ACM Transactions on Graphics* **41**(4), 1–15 (Jul 2022). <https://doi.org/10.1145/3528223.3530127>, <http://dx.doi.org/10.1145/3528223.3530127>
22. NVIDIA: Isaac Sim, <https://github.com/isaac-sim/IsaacSim>
23. Oquab, M., Darcet, T., Moutakanni, T., Vo, H., Szafraniec, M., Khalidov, V., Fernandez, P., Haziza, D., Massa, F., El-Nouby, A., Assran, M., Ballas, N., Galuba, W., Howes, R., Huang, P.Y., Li, S.W., Misra, I., Rabbat, M., Sharma, V., Synnaeve, G., Xu, H., Jegou, H., Mairal, J., Labatut, P., Joulin, A., Bojanowski, P.: Dinov2: Learning robust visual features without supervision (2024), <https://arxiv.org/abs/2304.07193>
24. OverTheReality: OverMaps-1K Dataset. Hugging Face Datasets (2025), https://huggingface.co/datasets/OverTheReality/OverMaps_1k
25. Pavlakos, G., Choutas, V., Ghorbani, N., Bolkart, T., Osman, A.A.A., Tzionas, D., Black, M.J.: Expressive body capture: 3d hands, face, and body from a single image (2019), <https://arxiv.org/abs/1904.05866>
26. Poole, B., Jain, A., Barron, J.T., Mildenhall, B.: Dreamfusion: Text-to-3d using 2d diffusion (2022), <https://arxiv.org/abs/2209.14988>
27. Puig, X., Undersander, E., Szot, A., Cote, M.D., Yang, T.Y., Partsey, R., Desai, R., Clegg, A.W., Hlavac, M., Min, S.Y., Vondruš, V., Gervet, T., Berges, V.P., Turner, J.M., Maksymets, O., Kira, Z., Kalakrishnan, M., Malik, J., Chaplot, D.S., Jain, U., Batra, D., Rai, A., Mottaghi, R.: Habitat 3.0: A co-habitat for humans, avatars and robots (2023), <https://arxiv.org/abs/2310.13724>
28. Radford, A., Kim, J.W., Hallacy, C., Ramesh, A., Goh, G., Agarwal, S., Sastry, G., Askell, A., Mishkin, P., Clark, J., Krueger, G., Sutskever, I.: Learning transferable visual models from natural language supervision (2021), <https://arxiv.org/abs/2103.00020>
29. Ramakrishnan, S.K., Gokaslan, A., Wijmans, E., Maksymets, O., Clegg, A., Turner, J., Undersander, E., Galuba, W., Westbury, A., Chang, A.X., Savva, M., Zhao, Y., Batra, D.: Habitat-matterport 3d dataset (hm3d): 1000 large-scale 3d environments for embodied ai (2021), <https://arxiv.org/abs/2109.08238>
30. Savva, M., Kadian, A., Maksymets, O., Zhao, Y., Wijmans, E., Jain, B., Straub, J., Liu, J., Koltun, V., Malik, J., Parikh, D., Batra, D.: Habitat: A platform for embodied ai research (2019), <https://arxiv.org/abs/1904.01201>

31. Shen, B., Xia, F., Li, C., Martín-Martín, R., Fan, L., Wang, G., Pérez-D’Arpino, C., Buch, S., Srivastava, S., Tchapmi, L.P., Tchapmi, M.E., Vainio, K., Wong, J., Fei-Fei, L., Savarese, S.: *igibson 1.0: a simulation environment for interactive tasks in large realistic scenes* (2021), <https://arxiv.org/abs/2012.02924>
32. SpatialVerse Research Team, M.T.I.: *InteriorGS: A 3d gaussian splatting dataset of semantically labeled indoor scenes*. <https://huggingface.co/datasets/spatialverse/InteriorGS> (2025)
33. Straub, J., Whelan, T., Ma, L., Chen, Y., Wijmans, E., Green, S., Engel, J.J., Mur-Artal, R., Ren, C., Verma, S., Clarkson, A., Yan, M., Budge, B., Yan, Y., Pan, X., Yon, J., Zou, Y., Leon, K., Carter, N., Briales, J., Gillingham, T., Mueggler, E., Pesqueira, L., Savva, M., Batra, D., Strasdat, H.M., Nardi, R.D., Goesele, M., Lovegrove, S., Newcombe, R.: *The replica dataset: A digital replica of indoor spaces* (2019), <https://arxiv.org/abs/1906.05797>
34. Szot, A., Clegg, A., Undersander, E., Wijmans, E., Zhao, Y., Turner, J., Maestre, N., Mukadam, M., Chaplot, D., Maksymets, O., Gokaslan, A., Vondrus, V., Dharur, S., Meier, F., Galuba, W., Chang, A., Kira, Z., Koltun, V., Malik, J., Savva, M., Batra, D.: *Habitat 2.0: Training home assistants to rearrange their habitat* (2022), <https://arxiv.org/abs/2106.14405>
35. Team, G., Anil, R., Borgeaud, S., et al.: *Gemini: A family of highly capable multimodal models* (2025), <https://arxiv.org/abs/2312.11805>
36. Wijmans, E., Kadian, A., Morcos, A., Lee, S., Essa, I., Parikh, D., Savva, M., Batra, D.: *Dd-ppo: Learning near-perfect pointgoal navigators from 2.5 billion frames* (2020), <https://arxiv.org/abs/1911.00357>
37. World Labs: *Marble: A multimodal world model* (11 2025), <https://www.worldlabs.ai/blog/marble-world-model>
38. Xiang, F., Qin, Y., Mo, K., Xia, Y., Zhu, H., Liu, F., Liu, M., Jiang, H., Yuan, Y., Wang, H., Yi, L., Chang, A.X., Guibas, L.J., Su, H.: *Sapient: A simulated part-based interactive environment* (2020), <https://arxiv.org/abs/2003.08515>
39. Yu, Z., Chen, A., Huang, B., Sattler, T., Geiger, A.: *Mip-splatting: Alias-free 3d gaussian splatting* (2023), <https://arxiv.org/abs/2311.16493>
40. Zhang, Y., Tang, S.: *The wanderings of odysseus in 3d scenes* (2022), <https://arxiv.org/abs/2112.09251>